Bringing SaC to the masses with sac4c

Jordy Aaldering

13 April 2026

The opinion of someone born years after SaC

1966
Bodo

1970
The Spanish Inquisition*

1994
SaC

1998
Me

Bodo's crystal ball

Single **A**ssignment **C** – Functional Programming Using Imperative Style

Sven-Bodo Scholz *

Functional programming languages did not yet find a broad acceptance by application programmers outside the functional community. The reasons for this situation are manifold and differ depending on the field of application. Of primary interest in this paper are scientific applications involving complex manipulations of arrays. For this class of applications the following criteria are the most important ones for the choice of a suitable language:

- availability of primitive and compound operations on arrays,
- efficiency of compiled code,
- readability of code,

Bodo's crystal ball

Single **A**ssignment **C** – Functional Programming Using Imperative Style

Sven-Bodo Scholz *

Functional programming languages did not yet find a broad acceptance by application programmers outside the functional community. The reasons for this situation are manifold and differ depending on the field of application. Of primary interest in this paper are scientific applications involving complex manipulations of arrays. For this class of applications the following criteria are the most important ones for the choice of a suitable language:

- availability of primitive and compound operations on arrays,
- efficie[illegible]
- readability of code,

HP3

Bodo's crystal ball

Single **A**ssignment **C** – Functional Programming Using Imperative Style

Sven-Bodo Scholz *

Functional programming languages did not yet find a broad acceptance by application programmers outside the functional community. The reasons for this situation are manifold and differ depending on the field of application. Of primary interest in this paper are scientific applications involving complex manipulations of arrays. For this class of applications the following criteria are the most important ones for the choice of a suitable language:

- availability of primitive and compound operations on arrays,
- efficient code,
- readability of code,
- and the integration of I/O (e.g. for a graphical representation of results).

HP3

Compiling with sac2c

mandelbrot.sac

mandelbrot

Compiling with sac2c

Exposing C to SaC

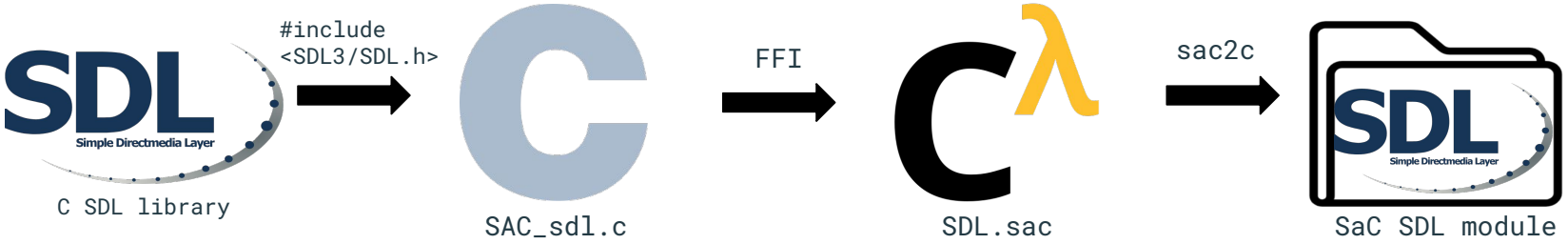
- How to visualise Mandelbrot in SaC?

tions involving complex manipulations of arrays. For this class of applications the following criteria are the most important ones for the choice of a suitable language:

- availability of primitive and compound operations on arrays,
- efficiency of compiled code,
- readability of code,
- and the integration of I/O (e.g. for a graphical representation of results).

Exposing C to SaC

- How to visualise Mandelbrot in SaC?
- We need some SaC SDL module
  - Expose C functionality to SaC

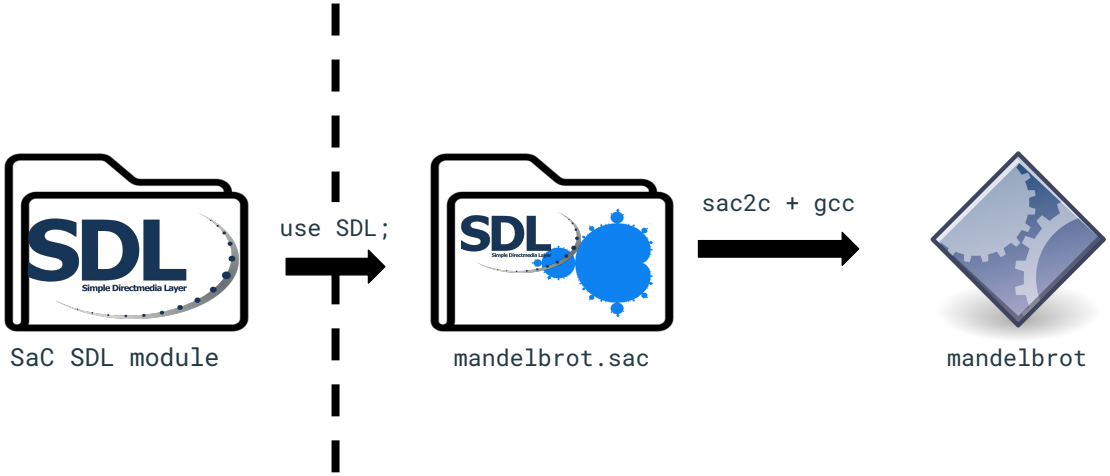


Most modules are external

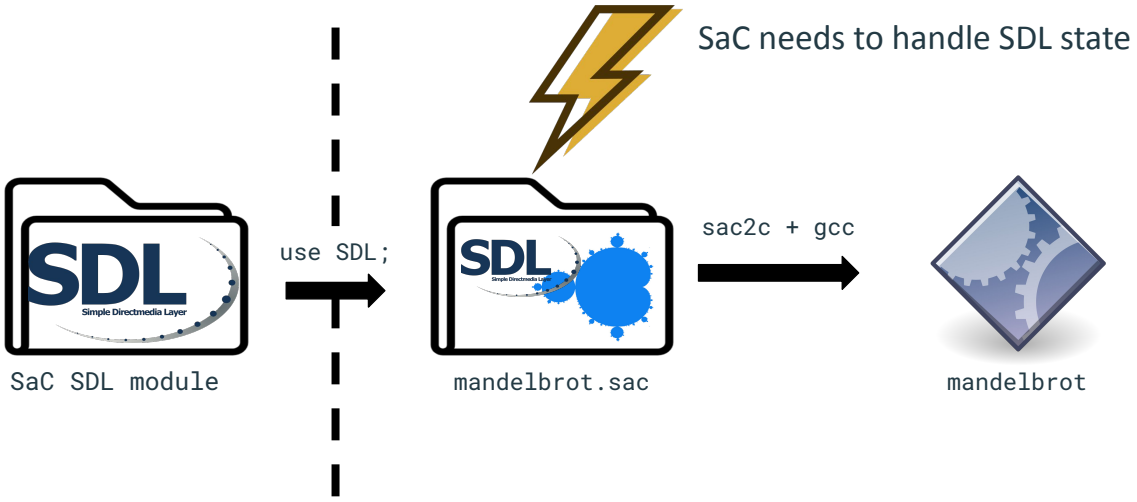
- StdLib *https://github.com/SacBase/Stdlib*
- SDL *https://github.com/SacBase/SDL*
- OpenCV *https://github.com/SacBase/OpenCV*
- Cuda *https://github.com/SacBase/CUDA*
- BLAS *https://github.com/SacBase/BLAS*

SaC SDL module

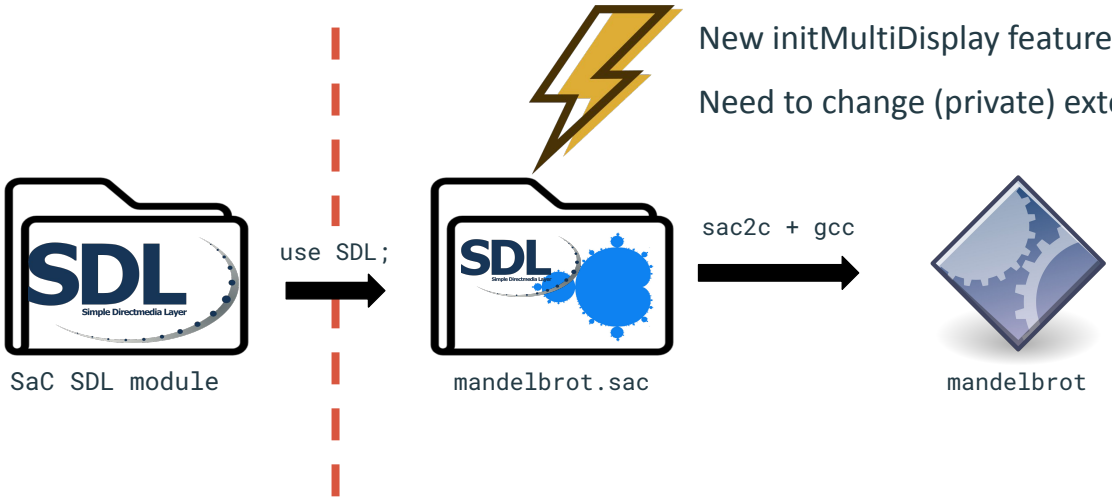
Using external modules



Using external modules



Using external modules



New initMultiDisplay feature needed?

Need to change (private) external module!

A lot of boilerplate

- Track state in SaC

```
class File;

external classtype; // file descriptor
```

A lot of boilerplate

- Track state in SaC
- FFI links in SaC

```
class File;

external classtype; // file descriptor

external syserr fclose(File fd);
  #pragma linkname “SAC_close”
  #pragma …
```

A lot of boilerplate

- Track state in SaC
- FFI links in SaC
- (Thin) wrappers in C

```
class File;

external classtype; // file descriptor

external syserr fclose(File fd);
  #pragma linkname “SAC_close”
  #pragma …
```

```
sac_int SAC_close(int fd) {
  int ret = close(fd);
  int err = ret == -1 ? errno : -1;
  return (sac_int)err;
}
```

A lot of boilerplate

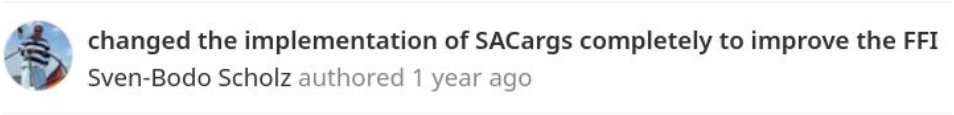
- Track state in SaC
- FFI links in SaC
- (Thin) wrappers in C
- Type conversions (and memory management)

```
class File;

external classtype; // file descriptor

external syserr fclose(File fd);
  #pragma linkname “SAC_close”
  #pragma …
```

```
sac_int SAC_close(int fd) {
  int ret = close(fd);
  int err = ret == -1 ? errno : -1;
  return (sac_int)err;
}
```

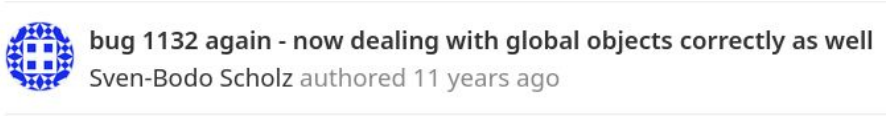
The cost of exposing C to SaC

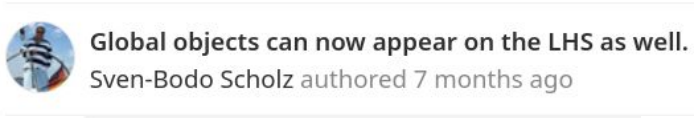
- Updating external dependencies a lot of work, or not even possible
- Memory management across languages is difficult
  - Lots of effort into `SACargs` to make this easier

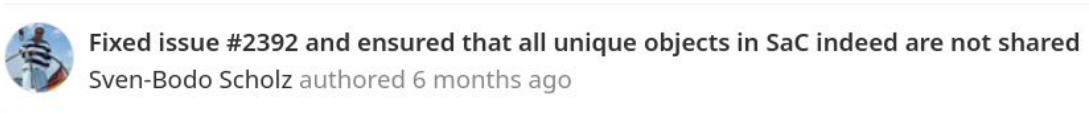


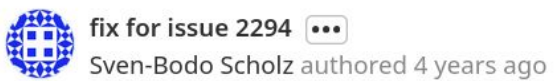
The cost of exposing C to SaC

- Updating external dependencies a lot of work, or not even possible
- Memory management across languages is difficult
  - Lots of effort into `SACargs` to make this easier
- Compiler needs additional machinery and maintenance
  - Global objects (like stdio) need to be woven into control flow to remain functional



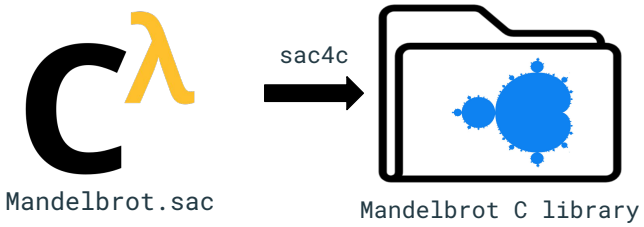




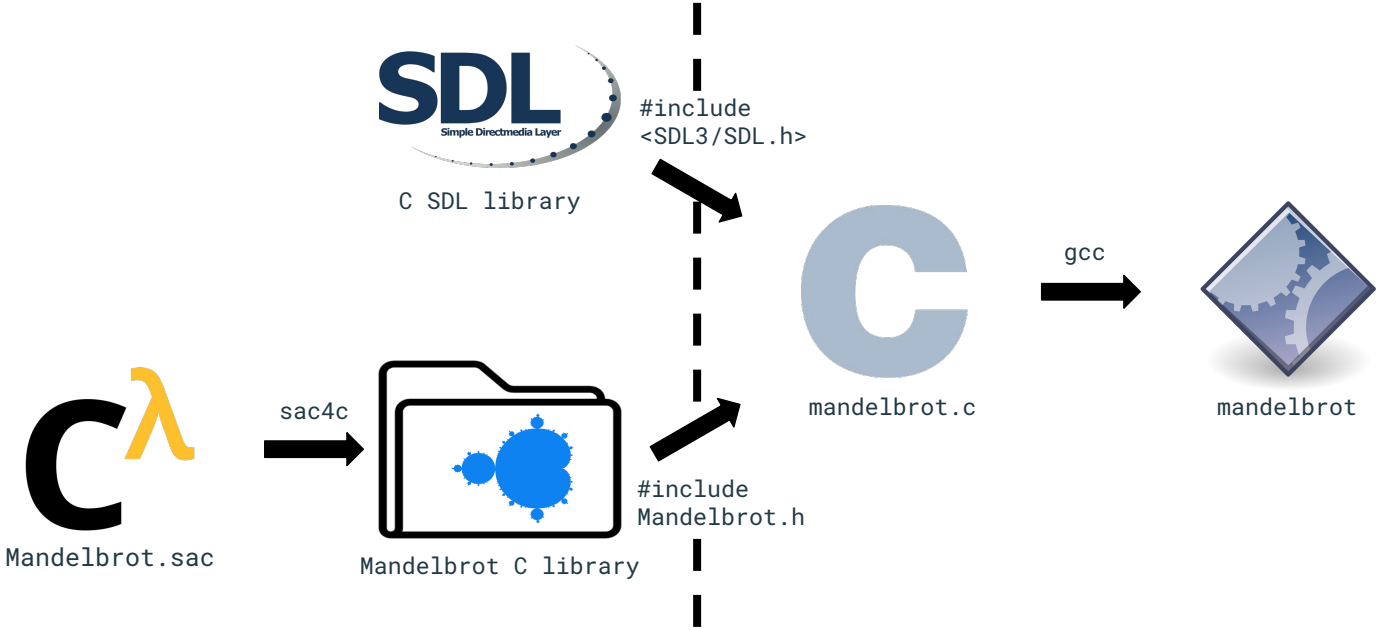


Compiling **for** C, instead of **to** C

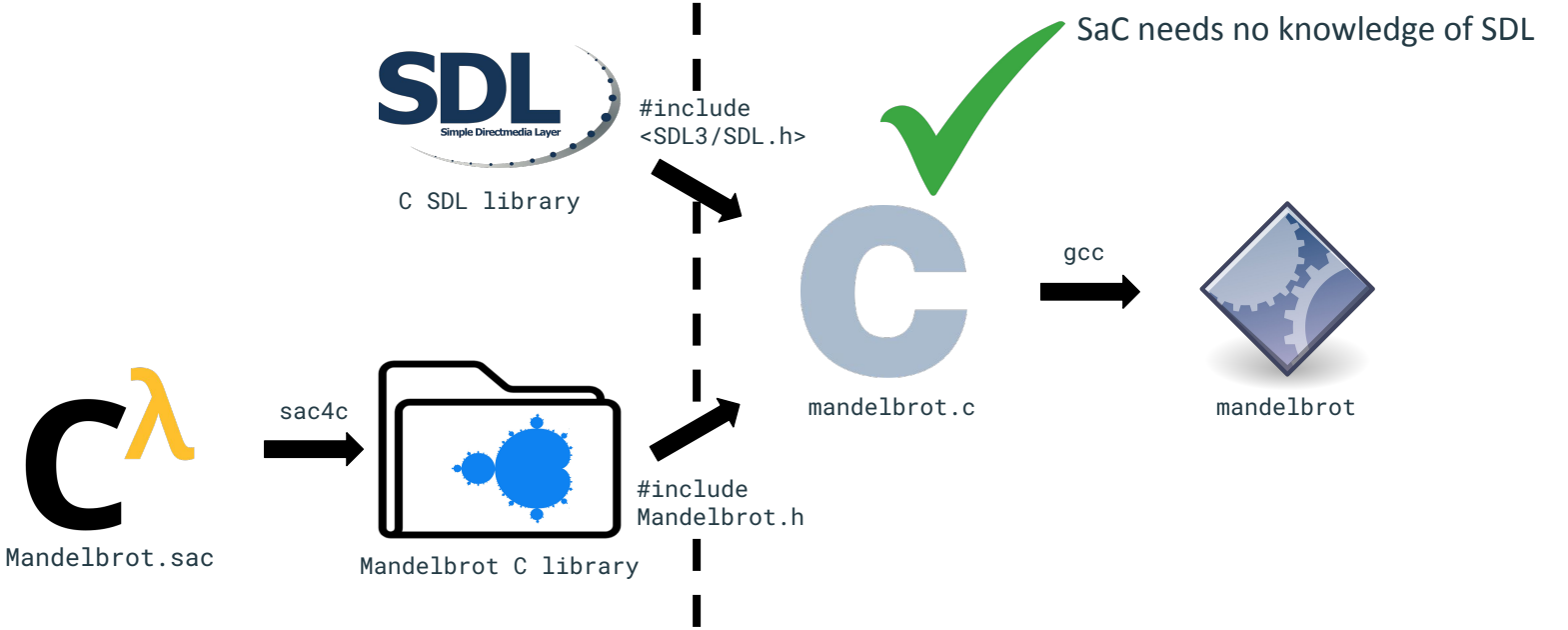
- **sac2c:** Compile SaC programs directly to binary (using C)
- **sac4c:** Compile SaC programs to C code, for use in other C programs



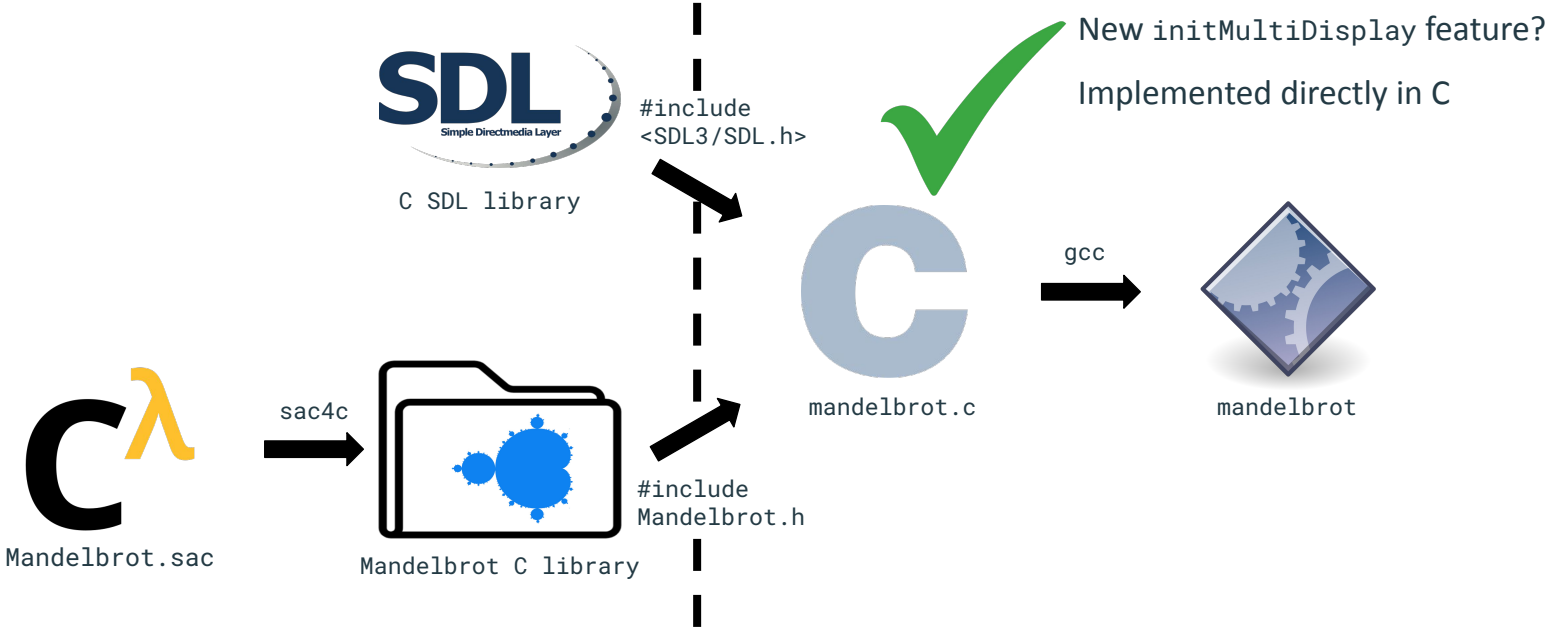
Using external modules



Using external modules



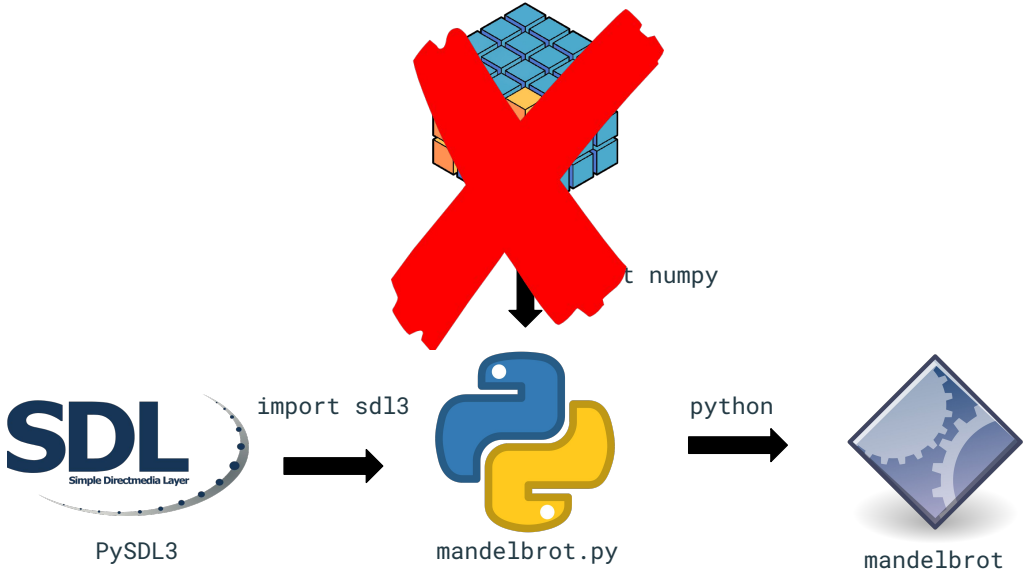
Using external modules



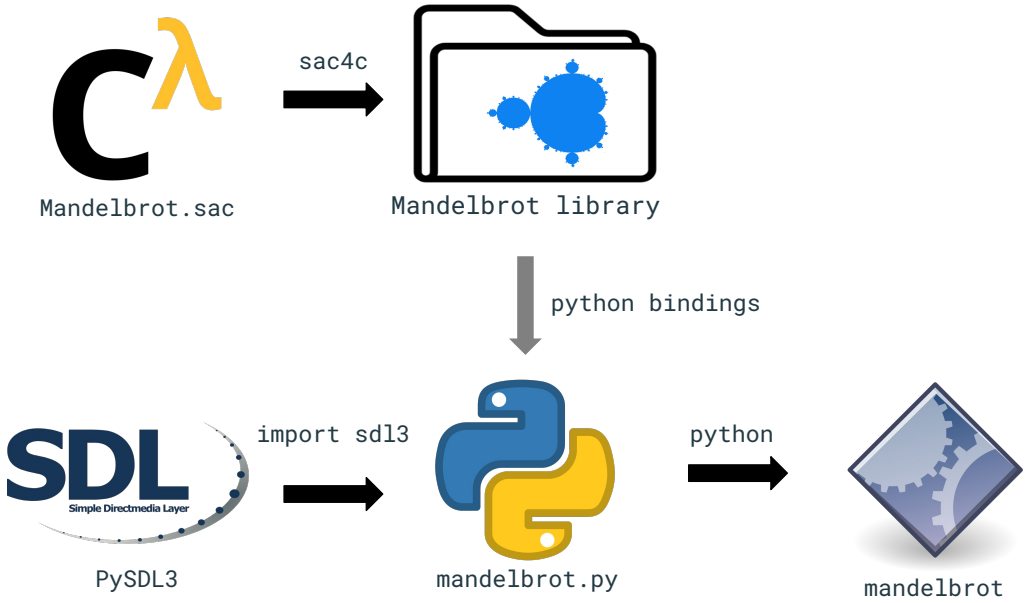
From C, we can go anywhere

- Everyone and their dog uses numpy
- Can we replace it with SaC?
- *Sorry Bodo for suggesting this heresy*

A future where everyone uses SaC



A future where everyone uses SaC



Limitations?

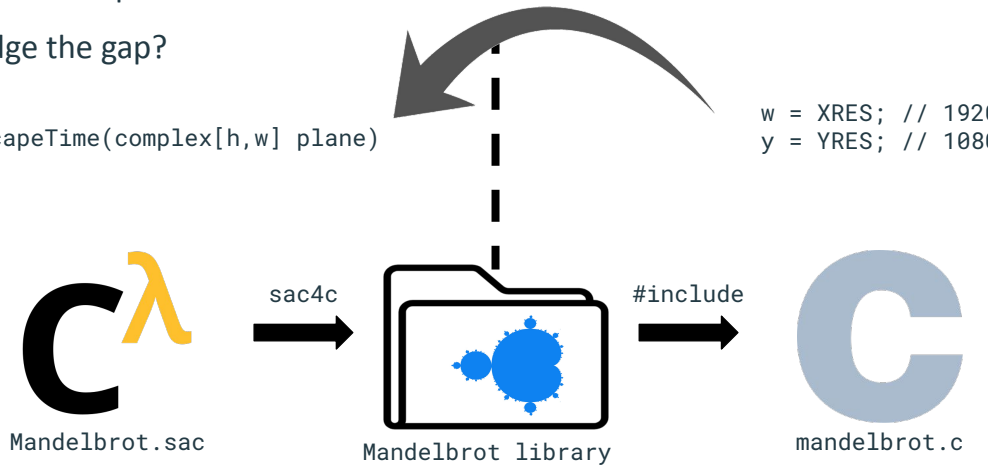
- Of course…

Specialisation

- Shape knowledge moved out of SaC code
- Limits optimisation potential
- How to bridge the gap?

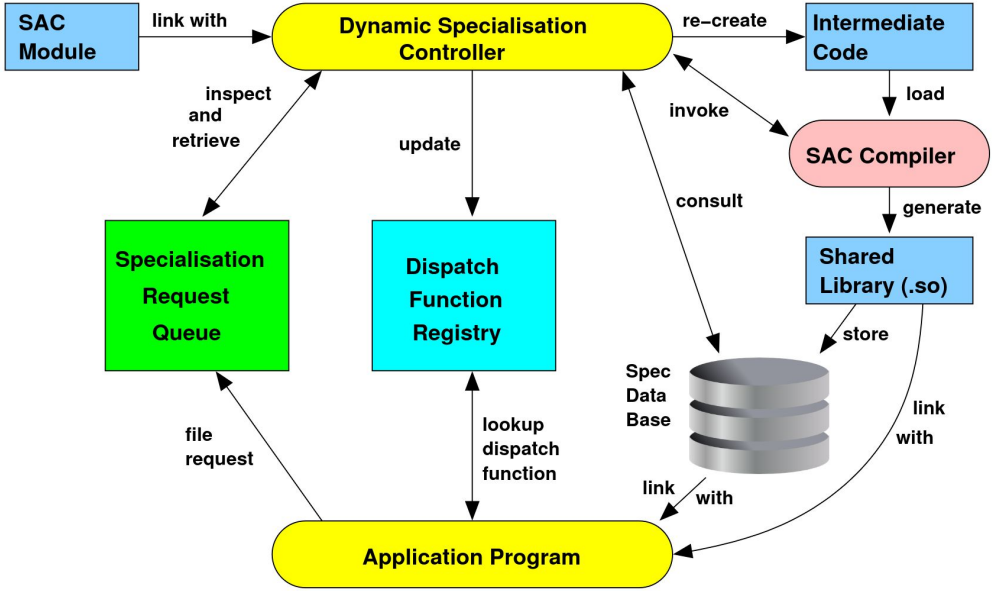
```
int[h,w] escapeTime(complex[h,w] plane)
```

```
w = XRES; // 1920
y = YRES; // 1080
```



Specialise on demand!

- Generate specialisations at runtime
- This is not easy



Grelck, C. and Wiesinger, H. *Next Generation Asynchronous Adaptive Specialization for Data-Parallel Functional Array Processing in SaC: Accelerating the Availability of Specialized High Performance Code*.

Alternate universe

- Making compiling *for C* practical requires a lot of implementation effort
- Where would SaC be today, had sac4c been the primary focus?

tions involving complex manipulations of arrays. For this class of applications the following criteria are the most important ones for the choice of a suitable language:

- availability of primitive and compound operations on arrays,
- efficiency of compiled code,
- readability of code,
- and the integration of I/O (e.g. for a graphical representation of results).

sac4c deserves more ❤️

- Split HPC code from I/O and state
- But, no free lunch
  - Limited (shape) knowledge may hurt performance
  - Will always need FFI bindings and type conversions
- Documentation?

Open **Demo 'sac_from_c' does not compile**

sac-group / sac2c #2209 · created 11 years ago